



# Estácio

## 2º Procedimento | Criação do Cadastro em Modo Texto

Nome da disciplina: Programação Back- end com Java

Campus: POLO ELDORADO III - CONTAGEM – MG

Curso: DESENVOLVIMENTO FULL STACK

Turma: DGT2821 – 9001

2026/1

Guilherme Rogério Souza Comunian

[Link do Repositório](#)

## Sumário

|                                                                                                             |   |
|-------------------------------------------------------------------------------------------------------------|---|
| Objetivo Geral .....                                                                                        | 3 |
| 1 - Alterar o método main da classe principal do projeto, para implementação do cadastro em modo texto..... | 4 |
| 2 - Ajustar as características para obter uma execução como a seguinte.....                                 | 7 |
| 3 - Verificar todas as funcionalidades solicitadas e os arquivos gerados.....                               | 7 |
| Análise e Conclusão .....                                                                                   | 8 |
| a) O que são elementos estáticos e qual o motivo para o método main adotar esse modificador? .....          | 8 |
| b) Para que serve a classe Scanner? .....                                                                   | 8 |
| c) Como o uso de classes de repositório impactou na organização do código? .....                            | 9 |

## Objetivo Geral

Desenvolver um sistema de cadastro em modo texto utilizando Java, aplicando conceitos de Programação Orientada a Objetos e manipulação de entrada de dados via console, permitindo a interação do usuário para operações de inclusão, alteração, exclusão e consulta, além da persistência e recuperação de dados em arquivos binários.

1 - Alterar o método main da classe principal do projeto, para implementação do cadastro em modo texto.

```
// ... //
int selectedOption;
char selectedTypePessoa = ' ';
PessoaFisicaRepo repoPF = new PessoaFisicaRepo();
PessoaJuridicaRepo repoPJ = new PessoaJuridicaRepo();
Scanner inptReader = new Scanner(System.in);

do {
    System.out.printf("Opcoes:%n "
        + "===== %n"
        + "1 - Incluir%n"
        + "2 - Alterar%n"
        + "3 - Excluir%n"
        + "4 - Exibir p/ Id%n"
        + "5 - Exibir Todos%n"
        + "6 - Salvar%n"
        + "7 - Recuperar Dados%n"
        + "0 - Exit.%n"
        + "===== %n");

    selectedOption = inptReader.nextInt();

    if(selectedOption < 0 || selectedOption > 7) {
        return;
    }

    switch (selectedOption){
        case 6:
        case 7:
            break;
        case 0:
            return;
        default:
            System.out.printf("F - Pessoa Fisica%n"
                + "J - Pessoa Juridica%n");

            selectedTypePessoa = inptReader.next().toUpperCase().charAt(0);

            if(selectedTypePessoa != 'F' && selectedTypePessoa != 'J') {
                System.out.println("Opcao invalida!");
                return;
            }
            break;
    }
}
// ... //
```

```

// ... //
switch (selectedOption){
    case 1:
        System.out.println("Id:");
        int id = inptReader.nextInt();
        System.out.println("Nome:");
        String nome = inptReader.next();

        if (selectedTypePessoa == 'F') {
            System.out.println("CPF:");
            String cpf = inptReader.next();
            System.out.println("Idade:");
            int idade = inptReader.nextInt();

            repoPF.insert(new PessoaFisica(id, nome, cpf, idade));
        } else {
            System.out.println("CNPJ:");
            String cnpj = inptReader.next();

            repoPJ.insert(new PessoaJuridica(id, nome, cnpj));
        }
        break;

    case 2:
        System.out.println("Id:");
        int idAlt = inptReader.nextInt();

        if (selectedTypePessoa == 'F') {
            PessoaFisica pf = repoPF.get(idAlt);
            if (pf != null) {
                pf.show();

                System.out.println("Novo Nome:");
                String nomeNovo = inptReader.next();
                System.out.println("Novo CPF:");
                String cpf = inptReader.next();
                System.out.println("Nova Idade:");
                int idade = inptReader.nextInt();

                repoPF.alter(new PessoaFisica(idAlt, nomeNovo, cpf, idade));
            }
        } else {
            PessoaJuridica pj = repoPJ.get(idAlt);
            if (pj != null) {
                pj.show();

                System.out.println("Novo Nome:");
                String nomeNovo = inptReader.next();
                System.out.println("Novo CNPJ:");
                String cnpj = inptReader.next();

                repoPJ.alter(new PessoaJuridica(idAlt, nomeNovo, cnpj));
            }
        }
        break;

    case 3:
        System.out.println("Id:");
        int idDel = inptReader.nextInt();

        if (selectedTypePessoa == 'F') {
            repoPF.delete(idDel);
        } else {
            repoPJ.delete(idDel);
        }
        break;
}
// ... //

```

```

// ... //
case 4:
    System.out.println("Id:");
    int idGet = inptReader.nextInt();

    if (selectedTypePessoa == 'F') {
        PessoaFisica pf = repoPF.get(idGet);
        if (pf != null) pf.show();
    } else {
        PessoaJuridica pj = repoPJ.get(idGet);
        if (pj != null) pj.show();
    }
    break;

case 5:
    if (selectedTypePessoa == 'F') {
        for (PessoaFisica p : repoPF.getAll()) {
            p.show();
        }
    } else {
        for (PessoaJuridica p : repoPJ.getAll()) {
            p.show();
        }
    }
    break;

case 6:
    try {
        System.out.println("Prefixo:");
        String prefixo = inptReader.next();

        repoPF.keep(prefixo + ".fisica.bin");
        repoPJ.keep(prefixo + ".juridica.bin");
    } catch (Exception e) {
        System.out.println("Erro ao salvar");
    }
    break;

case 7:
    try {
        System.out.println("Prefixo:");
        String prefixo = inptReader.next();

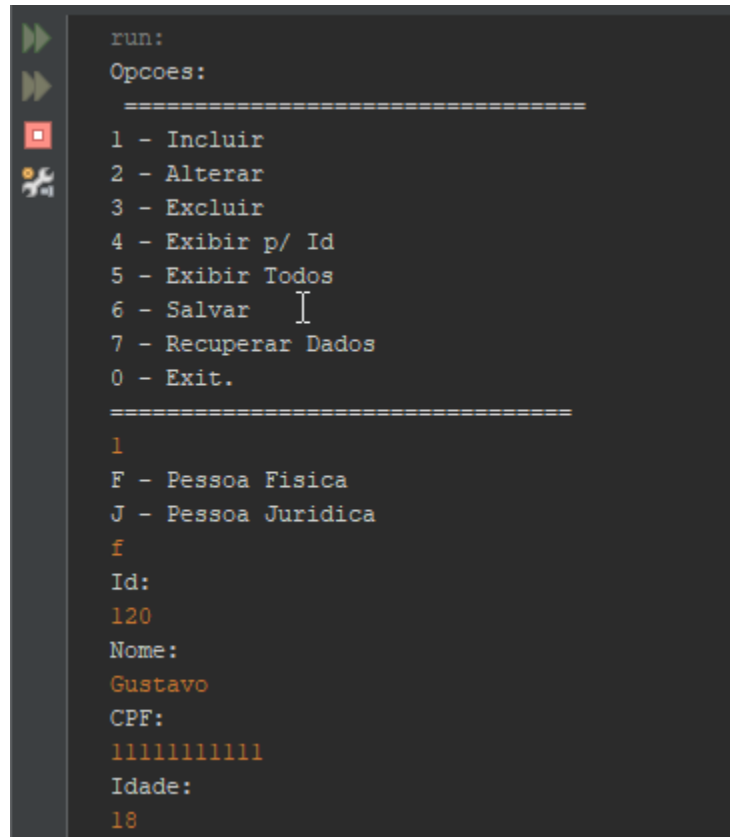
        repoPF.retrive(prefixo + ".fisica.bin");
        repoPJ.retrive(prefixo + ".juridica.bin");
    } catch (Exception e) {
        System.out.println("Erro ao recuperar");
    }
    break;
}

} while(selectedOption != 0);
// ... //

```

Precisei dividir o código em 3 partes, mas o arquivo se encontra integralmente no Github.

2 - Ajustar as características para obter uma execução como a seguinte.



```
run:
Opcoes:
=====
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir p/ Id
5 - Exibir Todos
6 - Salvar
7 - Recuperar Dados
0 - Exit.
=====
1
F - Pessoa Fisica
J - Pessoa Juridica
f
Id:
120
Nome:
Gustavo
CPF:
11111111111
Idade:
18
```

3 - Verificar todas as funcionalidades solicitadas e os arquivos gerados.

Todas as novas e antigas funcionalidades foram verificadas por meio da nova interface.

### Exemplo de teste, recuperação de dados.

```
7
Prefixo:
teste
Dados de Pessoa Fisica Recuperados
Dados de Pessoa Juridica Recuperados
Opcoes:
=====
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir p/ Id
5 - Exibir Todos
6 - Salvar
7 - Recuperar Dados
0 - Exit.
=====
4
F - Pessoa Fisica
J - Pessoa Juridica
f
Id:
120
Pessoa Fisica:
Id: 120
Name: Gustavo
Cpf: 11111111111
Age: 18
```

## Análise e Conclusão

### a) O que são elementos estáticos e qual o motivo para o método main adotar esse modificador?

Elementos estáticos são aqueles pertencentes à classe e não a uma instância específica, sendo carregados uma única vez na memória e compartilhados entre todos os objetos. O método main é estático porque ele precisa ser executado sem a necessidade de instanciar a classe, funcionando como ponto de entrada da aplicação.

### b) Para que serve a classe Scanner?

A classe Scanner é utilizada para realizar a leitura de dados de entrada, como informações digitadas pelo usuário no console. Ela permite capturar diferentes tipos de dados, como inteiros, strings e caracteres, facilitando a interação entre o usuário e o sistema.



c) Como o uso de classes de repositório impactou na organização do código?

O uso de classes de repositório contribui para a organização do código ao separar a lógica de manipulação dos dados da lógica principal da aplicação. Isso facilita a manutenção, reutilização e escalabilidade do sistema, além de seguir boas práticas de desenvolvimento como o princípio da responsabilidade única.